Cross-Checking Windows Artifacts to Detect Anti-Forensic Tampering

Captain Scott Russell (CYBSC 5360)
Department of Computer Engineering (M.S. - Cybersecurity)
Iowa State University, Ames, Iowa russesco@iastate.edu

Abstract: **People hiding evidence on a computer can change a file's dates, rename a photo so it looks like a program, or mask one file inside another. But Windows and its file system write the same facts down in several different places, and a cover-up rarely changes all of them! I built the Artifact Disagreement Scorer (ADS), a tool that compares these records with each other and points the examiner to the files where they disagree, with a plain-language reason for each one.**

ADS runs as a command-line program or as a single web page that works offline. I have tested it on the two disk images from this course (Project #1 and Project #2), using my own findings from EnCase and the Forensic Toolkit (FTK) as the expected answer key. On Project 1 it ranked six of the seven evidence files in its top ten out of 2,246 files. On Project 2, it found seven of the sixteen evidence files among 4,818, five of them in its top ten.

Keywords: **Digital forensics, anti-forensics, timestamp tampering, NTFS, triage, tool validation**

I. THE PROBLEM

Hiding evidence is cheap. Any program can rewrite the dates Windows shows for a file, a trick called *timestomping* [1]. A photo can be renamed *gzip* so it sits among system programs, and a file can be hidden inside a ZIP or after the end of another file.

Hiding it *well* is much harder. Windows records the same facts in more than one place, written by different parts of the system for different reasons. Windows' file system (NTFS) keeps a second, harder-to-reach copy of every file's dates. Windows writes a shortcut file when a document is opened, a Prefetch file when a program runs, and event logs for logons and clock changes. A file's name says what it should be, and its first bytes say what it really is encoded as. Someone covering their tracks may fix one of these, but rarely all of them, so a cover-up leaves records that *disagree* with one another.

Today's tools leave that comparison to the examiner. EnCase and the Forensic Toolkit display every record, but the examiner has to notice that two of them conflict. Specialized tools automate a single check, and a single check is noisy. Normal use also produces odd dates, so one filter can return hundreds of files as false positives.

I come to this problem with a unique perspective. For seven years I have served as an Air Force Offensive Cyberspace Operations Officer, more recently as a Cyber Operator with the 367th Cyberspace Operations Squadron at Joint Base San Antonio-Lackland, Texas, supporting U.S. Cyber Command and the National Security Agency. Currently, I am an Assistant Professor in the Air Force Aerospace Studies department at Iowa State University. An operator plans around the traces they leave, but by Locard's exchange principle, every contact leaves a trace. My military operator training, the GIAC Certified Forensic Analyst (GCFA) certification, and this department's CPRE 4360 course showed me where those traces live. This paper asks whether a tool can do that comparison quickly for the examiner and speed up the first stage of an analysis.

The question for this paper is the one from my topic proposal: *does combining many disagreement checks point an examiner to hidden evidence better than any single check?* To answer it I built the **Artifact Disagreement Scorer**. Fig. 1 shows the front page of its report on the Project 2 image: of 5,607 entries on the drive, 164 files have at least one clue and 7 are marked "look first."



Fig. 1. The Artifact Disagreement Scorer's report on the Project 2 image narrows the whole drive to a short list.

Contributions. The feedback on my topic proposal suggested surveying the literature first, aiming for a dataset other researchers can use, and looking for the weaknesses of existing solutions before building a new one. This paper answers each suggestion:

1. **A survey of current work.** Section II reviews the existing single checks and the work on combining records, and names the open problem this paper tests.
2. **A new solution.** The Artifact Disagreement Scorer combines many checks into one explained ranking, adds

two new date rules, and weighs each record by how hard it is to fake (Section III).

3. **Weaknesses of existing checks.** The visible-versus-protected date check that commercial tools rely on flagged almost every file on a drive whose dates had been reset in bulk, so on its own it pointed at nothing; and no single check found more than four of Project 1's seven evidence files (Section IV).
4. **A reusable dataset.** The answer keys for both course drives, synthetic Windows records with planted tampering and their ground truth, a small synthetic drive image, and seeded generators for stress-test drives all ship with the tool, so others can test their own methods against the same data. The course drive images belong to the course and are not redistributed.

Section II covers what already exists, Section III shows how the tool works, and Section IV explains how I am testing it.

II. WHAT ALREADY EXISTS

Two copies of every date. Windows' file system keeps a table with one entry per file (the Master File Table). Each entry holds four dates, for when the file was created, last changed, last opened, and when its entry last changed. It holds them *twice*. The **visible dates** (formally, Standard Information) are what Windows Explorer shows, and any program can change them. The **protected dates** (formally, File Name) are kept by the file system itself and cannot be set the normal way. When the visible dates say a file is older than the protected dates do, something is off: either the dates were rewritten, or the file was copied in a way that reset them.

Single checks. Chow and colleagues listed the rules that these dates follow in normal use [2]. Willassen [3] showed that the order in which files were laid down on disk, like layers of rock, can expose a file that claims to be older than its neighbors. Magnet AXIOM, a commercial tool, flags files whose visible and protected dates disagree [4], which is just one of my tool's checks on its own. EnCase and the Forensic Toolkit compare a file's name with its first bytes, another single check. Timeline tools put records in time order but do not test whether they agree.

Combining records. The closest work is Palmbach and Breitinger [5]. They showed that the change journal, shortcut files, Prefetch and event logs all keep traces of timestamp changes, concluded that none is reliable alone, and proposed five rules that use them together. Vanini, Hargreaves and Breitinger [6] proposed a way to rate how hard each *type* of record is to fake.

What is missing. Each of these stops short in a way that matters to an examiner:

- Palmbach and Breitinger [5] recommend combining sources but never measure whether combining helps. Their test used four files on an unused computer, so it cannot say how often ordinary activity sets off their rules.
- Vanini and colleagues [6] rate record types in general, not whether a particular drive shows tampering.
- The single checks are each easy to dodge or to trip by accident, and the commercial tools leave it to the examiner to connect them.

No one has measured, on the kind of images an examiner actually receives, whether combining these checks finds planted evidence faster than the best single check. That is the gap my tool is built to test.

III. HOW THE TOOL WORKS

A. Overview of the Tool

The **Artifact Disagreement Scorer** opens an exact copy of a drive without changing it, lines up each file's records, asks two sets of questions, and ranks the files by how many independent questions disagree about them (Fig. 2). Four goals shape it:

1. **Combine independent findings.** A file rises only when several independent checks disagree about it, so normal noise in one record does not put it at the top.

2. **A missing record is a clue.** Deleting a record is often easier than faking one [5], so the tool reports records that were cleared or are missing instead of quietly skipping the checks that need them.

3. **Explain every flag.** Each flag comes with a plain sentence and the values behind it, so an examiner can state and defend why a file was picked.

4. **Usable by non-specialists.** Findings go to investigators, lawyers, commanders and juries. The tool runs from the command line or as one web page that works offline, and its report is written in plain English so that a reader without technical training can follow every finding. In court that matters more than technical detail: a jury has to understand a finding to be an informed juror.

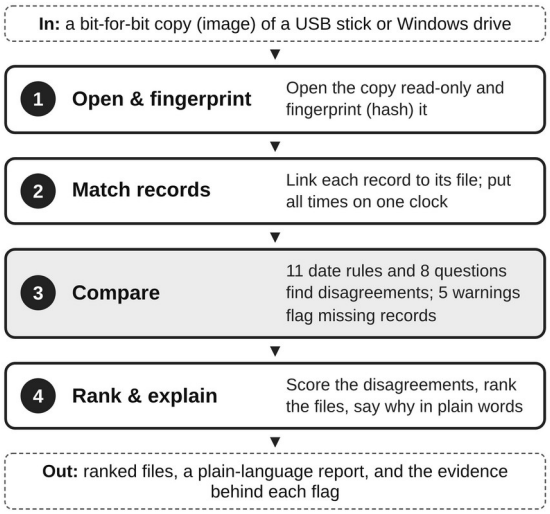


Fig. 2. How the tool works. Every input is fingerprinted, so each result traces back to the original drive copy.

Scope. Copies of USB sticks (the FAT file system) and Windows drives (the NTFS file system). The date rules target changes made from inside a running Windows system; editing a raw disk offline bypasses Windows' own logs and is therefore outside the scope of this project.

B. Where the Analysis Starts: the Drive Itself

Before any file is judged, the report shows what the drive says about itself (Fig. 3). The first sector of every drive, the *boot sector*, describes its layout: how big a sector is, how many there are, and where the file table sits. The tool decodes it field by field. In the tool, hovering over a field lights up its bytes, shows how they are read (numbers are stored with their bytes in reverse order), and marks the same field in the list on the right.

The rows of start-up code are folded away, and one click shows all 512 bytes. This gives the examiner a reference point before the leads, and a quick way to confirm the tool read the drive correctly.

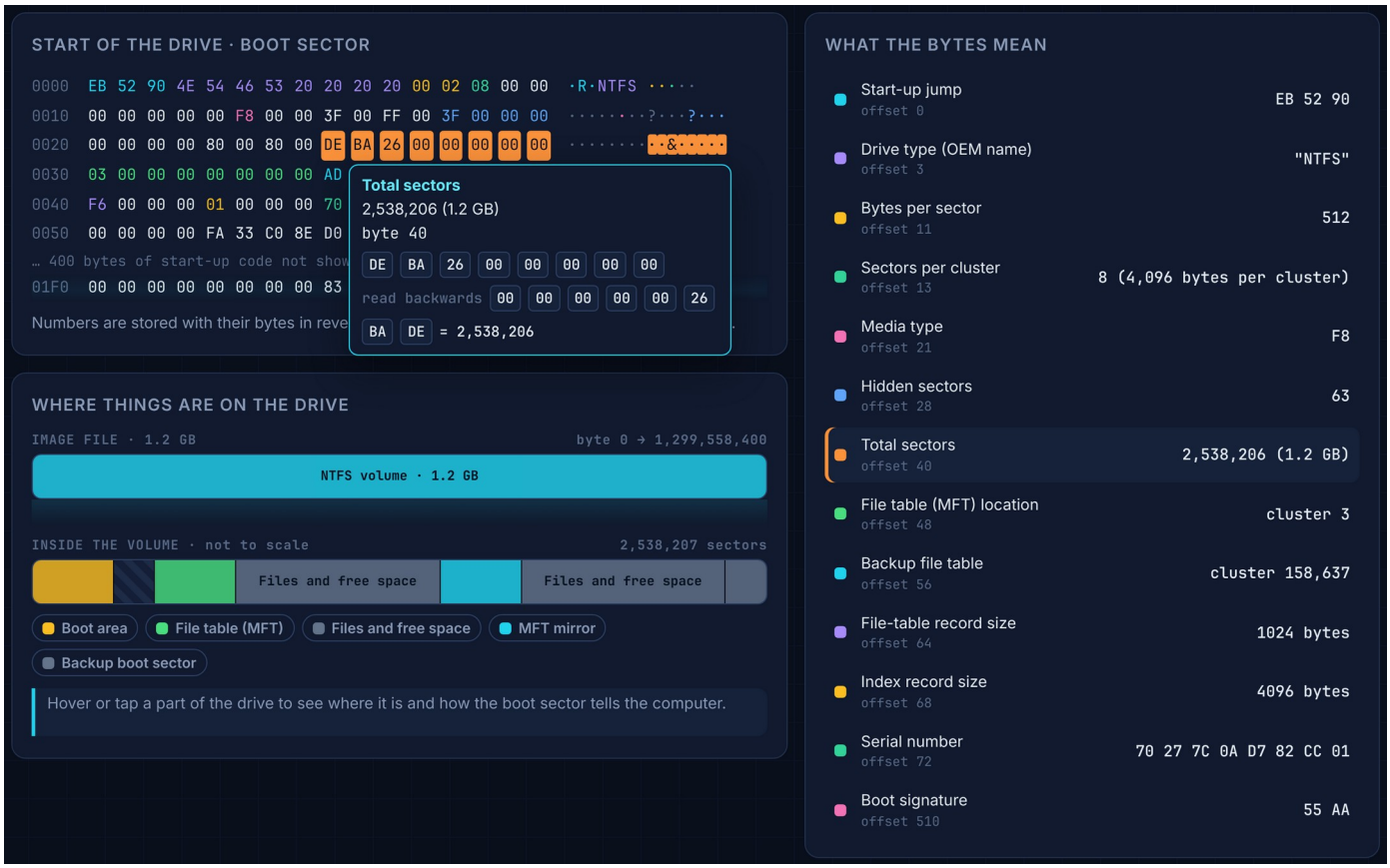


Fig. 3. The report's Drive tab on the Project 2 image. Hovering "Total sectors" lights up its eight bytes, shows them read in reverse order (2,538,206 sectors, 1.2 GB), and highlights the same field on the right; the map below shows where each part of the drive lies.

Fig. 4 shows one card from the browser edition's live view as it reads the Project 2 drive on my own computer. The drive is drawn as a grid of squares, one for each small slice of the disk. A square lights up when a file stored there is read, colored by what kind of file it is, and the counters keep a running total.

C. Eight First-Look Questions, for Any Drive

Many drives an examiner receives are not Windows system drives: a USB stick, or a data drive full of another system's files. So the tool first asks eight questions of every file, each comparing what a file claims to be with what it holds. Fig. 5 shows them as the report lists them, with how many files each one flagged on Project 2. For example, *disguised file* (C1) compares a file's name with its first bytes; *stands out from its neighbors* (C5) finds the only photo in a folder of programs; *never mentioned* (C6) finds a file that the folder's own guides and web pages never name, though they name its neighbors; and *changed at an unusual time* (F1) finds a file written apart from the bulk copy around it.

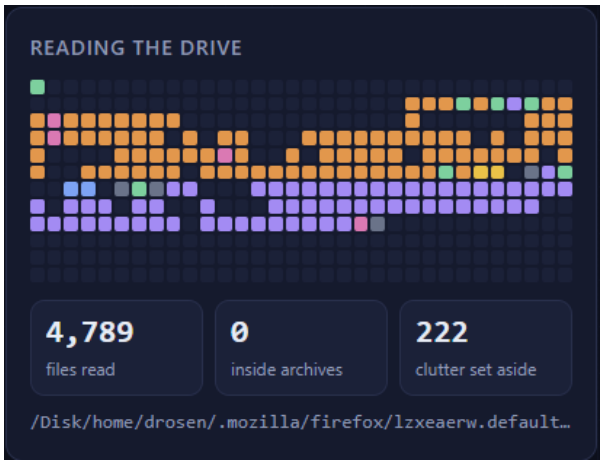


Fig. 4. The browser edition after reading the Project 2 drive. Each square is a slice of the disk, colored by the kind of file found there; gray squares are clutter set aside, and dark squares held no files.

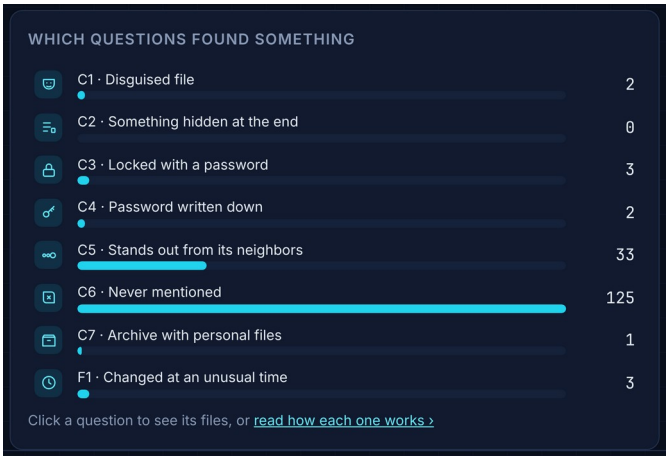


Fig. 5. The eight first-look questions as the report shows them, with how many Project 2 files each flagged.

D. Eleven Date Rules for Windows Drives

Windows keeps five records that the date rules compare (Table I). Each is written by a different part of Windows, which is what makes their agreement meaningful.

TABLE I
THE RECORDS THE DATE RULES COMPARE

Record	Written by	What the tool uses
Master File Table	File system	Visible and protected dates (8 per file)
Change journal	File system	When files were created and their dates changed
Shortcut files	Windows Explorer	Their own dates, and a copy of the target's dates
Prefetch files	Windows	Which programs ran, and their last 8 run times
Event logs	Windows	Logons, log clearing, clock changes

Table II lists the rules in the same plain words the report uses. Each rule answers *consistent*, *violated*, or *cannot check* when a record it needs is missing.

TABLE II
THE DATE RULES

Rule	Flags a file when...	From
R1	Its record-change date is before the journal's own entry	[5]
R2	A shortcut disagrees on its last-change date	[5]
R3	A shortcut is older than the file it points to	[5]
R4	A shortcut was opened after the file's last-opened date	[5]
R5	It changed while nobody was logged on	[5]
R6	Its visible date is older than the protected copy	[4]
R7	Its date was set to a round number of seconds	Known
R8	It was created later than its visible date says	New
R9	A shortcut remembers a different creation date	New
R10	Its date changed as an unusual program ran	[5] ext.
R11	It claims to be much older than its neighbors on disk	[3]

R8 and R9 are the main new rules. The change journal's "file created" entry is not touched when someone later rewrites the visible dates, so a journal entry dated well after the claimed creation date is strong evidence of back-dating. Every shortcut also keeps a copy of its target's dates, so a creation date that has since moved is suspicious even when the shortcut's own dates look normal. Five warnings (S1 to S5) keep missing records visible: a cleared event log, a deleted or late-starting change journal, gaps in the event log numbering, a changed system clock, and programs that ran without leaving a Prefetch file. Any "consistent" answer that rests on a damaged record is marked low-confidence.

E. The Scoring System

Every file gets a score between 0 and 1. Think of each check as a witness: the score is the share of witnesses that say something is wrong with the file. Not every witness counts the same:

- Rare clues count more. A check that flags almost every file on the drive tells us little, so it counts for very little; a check that flags only a handful of files counts for a lot.
- Hard-to-fake records count more. For the date rules, records a user cannot easily reach (the change journal, the protected dates) count up to three times more than records a user can edit or delete (shortcuts, Prefetch files), using the tamper-resistance ratings of Vanini and colleagues [6]. As far as I know, this is the first time those ratings have been used as numbers inside a detection tool.
- Checks that cannot run are left out, so a file is neither helped nor hurt by a record that is missing.

$$D(f) = \Sigma_r \, w_r \, v_r(f) \;/\; \Sigma_r \, w_r \qquad (1)$$

Equation (1) puts this together: $v_r(f)$ is 1 if check r flags file f and 0 if not, and each weight is $w_r = \ln(1/p_r)$, where p_r is the share of files that check r flags; the rarer the clue, the bigger the weight. A simple weighted score was chosen over machine learning for two reasons: there is no labeled training data, and every point of a file's score can be traced back to a named check. Fig. 6 shows the result on Project 2: the files ranked by score.

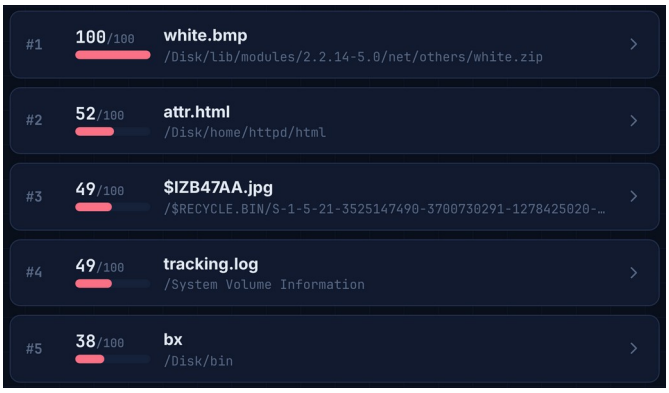


Fig. 6. The top five leads on the Project 2 image, ranked by score.

F. Every Flag Explained

Opening a lead shows why it was picked (Fig. 7). In this example, three independent questions agree about one file: its name says web page but its first bytes (FF D8 FF) say photo; it is the only photo in its folder; and the drive's guides and web pages name three of its four neighbors but never this file. Each line is a sentence an examiner can repeat in a report, with the evidence printed under it.



Fig. 7. One lead opened: each check that fired is explained in a plain sentence, with the evidence under it.

G. Implementation

The tool is written in Python 3 using only its standard library, so it runs on any computer with Python installed. It reads the drive copies itself and never writes to them. Both of its drive readers were checked against The Sleuth Kit, a standard open-source forensic toolkit: the USB-stick reader matched it byte for byte on every file of the Project 1 image. Every input is fingerprinted with SHA-256. The same code produces the "one check only" results that the evaluation compares against. A browser edition puts the whole tool in a single offline web page, and it gives exactly the same results as the command line on both course images. During development, a suite of automated tests checked the code for bugs.

IV. HOW I AM TESTING IT

The test is simple: on drives I did not build, does the tool point an examiner to what a careful manual examination found, and how much reading does that take?

A. Four Questions

1. **Does it point to the evidence?** What share of the evidence files I found by hand does it list, and how far down?
2. **Is combining better?** For the same number of files reviewed, does the combined ranking reach more evidence than any single check alone?
3. **Does it hold up on a drive it was not tuned on?**
4. **Can a non-specialist follow it?** Does the report alone explain each flag in a way a non-technical audience can understand?

B. The Two CYBSC 5360 Project Drives

Table III summarizes the two drives. I analyzed both by hand for the course assignments, without the tool, so each has an independent answer key.

TABLE III
THE TWO COURSE DRIVES

	Project 1	**Project 2**
Drive	External drive (FAT16, as on USB sticks)	Windows-formatted (NTFS) volume of copied Linux files
Examined by hand with	EnCase	Forensic Toolkit
Answer key	7 evidence files, 2 decoys	16 evidence files, 3 decoys

Answer keys. For each drive I listed the files that answered a case question (the evidence) and the files planted to mislead (decoys), with how I found each by hand. A file the tool lists that is not in the key is not necessarily a false alarm: the key is one examiner's findings.

Locking the checks. I wrote the eight first-look questions after examining Project 1, so its result is a demonstration, not a test. The questions were committed on October 20, before I examined Project 2, and an automated test fails if they change. That makes Project 2 a fair, held-out test. Reading Windows-formatted drives was added later, but it only reads the drive and changes no question.

Date rules on real data. Project 2 uses Windows' file system, so the three date rules that need only the file table (R6, R7, R11) run on it directly. It holds Linux files with no shortcuts, Prefetch files or Windows event logs, so the other rules are tested on made-up cases instead. There is no answer key for date tampering on this drive, so this part reports what the date rules found.

C. What I Measure

For each drive: how many evidence files the tool lists and where each one ranks; how many are in the top ten; how many files an examiner must read to reach every evidence file it found (the *review depth*); and how many decoys it lists. To compare with single checks, each check gets the same budget: if a check flags 12 files, it is compared with the tool's top 12.

D. Results So Far

Project 1 (development). The tool found all seven evidence files, six of them in its top ten, out of more than 2,000 files on the drive, while no single check found more than four. Combining also beat the broad checks head to head: the unusual-time check flags 12 files and catches 4 pieces of evidence, while the tool's top 12 hold 6. Its one weak spot was its very top file, a decoy, so the two narrowest checks, which each flag a single file, did better when only one file is read.

Project 2 (held-out test). This is the real test, run with the questions locked. Out of 4,818 files, the tool found 7 of the 16 evidence files, 5 of them in its top ten. That is less than half, short of my first success criterion, and again the narrowest single checks did better when only the top one to three files are read. Every miss has a clear reason: the evidence sat somewhere the tool does not look yet, such as a document's properties, the text inside a page, or a password typed as "pswd =". The whole first look took about half a minute, both from the command line and in the browser edition, which is easier to set up and has a cleaner interface.

The date rules found something different: a pattern across the whole drive (Fig. 8). On almost every file, the visible date was older than the protected one and set to a round number of seconds. Nearly every visible date had been reset to a single moment in October 2011, while the protected dates show the files were really written in two batches six years apart. A clue that fires on every file cannot single out any one of them, so the scoring gives these rules almost no weight; instead, the report raises them as a finding about the drive as a whole.

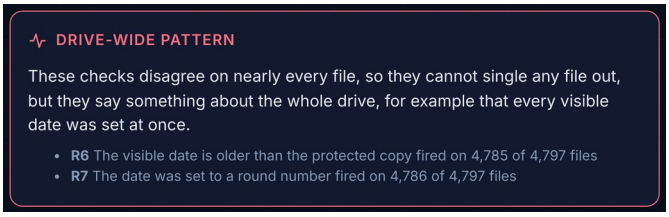


Fig. 8. The date-check report on Project 2: two rules disagree on almost every file, a sign that the whole drive's visible dates were reset at once.

E. Success Criteria (Set Before Project 2)

On Project 2: (1) the tool lists at least half of the evidence files; (2) for every single check, the tool's top files hold at least as many evidence files as that check does with the same budget; (3) every top-twenty flag is explained by the report alone; and (4) every missed evidence file is explained by what the tool does and does not examine. Criteria 1 and 2 were not met, and that is reported as found. Criteria 3 and 4 appear met.

F. Limits of This Test

Two drives are a very small sample, too small for statistical conclusions. Each answer key is one examiner's findings and may be incomplete. Both drives are teaching cases with planted evidence, not real adversaries. Having worked on the offensive side, I expect a capable adversary who knows which records a tool compares to try to fix all of them; that is why the tool also treats a cleared or missing record as a clue rather than a dead end. I wrote the checks myself, which is why Project 1 counts only as development. The Windows-only date rules meet real data only through the file table; the rules that need shortcuts, Prefetch and event logs are tested on made-up cases.

G. Status and Next Steps

Done: the tool, both drive readers, the answer keys, and the runs on both projects. The Project 1 result reproduced exactly on a second computer, and the browser edition matched the command line on both drives. For the final paper, I will revise this report based on feedback from the instructor and TA, and make the tool's interface and prompts more intuitive.

DISCLOSURE OF AI USE

I used Anthropic's Claude large language model (Opus 5.5) as a coding assistant to build the Artifact Disagreement Scorer. It worked from specifications I wrote for the project's scope and from the outline in my one-page topic proposal, and it produced the skeleton of the code and the visual design of the reports and the browser edition. I tested every part of the tool myself, and the answer keys come from my own manual examinations of both drives. Claude also drafted and revised parts of this paper's text at my direction, made the figures from my tool's screenshots, and formatted the paper to the template on Canvas. Iowa State University's guidance [7] asks users to review AI-generated content before using it and to cite the use of generative AI; I reviewed all of the AI-assisted code, text and figures, and I take responsibility for the accuracy of this paper.

REFERENCES

[1] MITRE, "Indicator Removal, Technique T1070," MITRE ATT&CK. [Online]. Available: https://attack.mitre.org/techniques/T1070/

[2] K.-P. Chow, F. Y. Law, M. Y. Kwan, and P. K. Lai, "The rules of time on NTFS file system," in *Proc. 2nd Int. Workshop Systematic Approaches to Digital Forensic Engineering (SADFE)*, 2007, pp. 71–85.

[3] S. Y. Willassen, "Finding evidence of antedating in digital investigations," in *Proc. 3rd Int. Conf. Availability, Reliability and Security (ARES)*, 2008, pp. 26–32.

[4] R. Andrade, "Expose evidence of timestomping with the NTFS timestamp mismatch artifact," Magnet Forensics, 2020.

[5] D. Palmbach and F. Breitinger, "Artifacts for detecting timestamp manipulation in NTFS on Windows and their reliability," *Forensic Sci. Int.: Digit. Investig.*, vol. 32, 300920, 2020.

[6] C. Vanini, C. Hargreaves, and F. Breitinger, "Evaluating tamper resistance of digital forensic artifacts during event reconstruction," arXiv:2412.12814, 2024.

[7] Iowa State University, "Policies and Compliance," Artificial Intelligence, Iowa State University. [Online]. Available: https://ai.iastate.edu/policies-and-compliance